

Verification Continuum™

IDX Assertion Verification Acceleration User Guide

Version W-2024.09, September 2024



Copyright and Proprietary Information Notice

© 2024 Synopsys, Inc. This Synopsys software and all associated documentation are proprietary to Synopsys, Inc. and may only be used pursuant to the terms and conditions of a written license agreement with Synopsys, Inc. All other use, reproduction, modification, or distribution of the Synopsys software or the associated documentation is strictly prohibited.

Destination Control Statement

All technical data contained in this publication is subject to the export control laws of the United States of America. Disclosure to nationals of other countries contrary to United States law is prohibited. It is the reader's responsibility to determine the applicable regulations and to comply with them.

Disclaimer

SYNOPSYS, INC., AND ITS LICENSORS MAKE NO WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, WITH REGARD TO THIS MATERIAL, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE.

Trademarks

Synopsys and certain Synopsys product names are trademarks of Synopsys, as set forth at <https://www.synopsys.com/company/legal/trademarks-brands.html>. All other product or company names may be trademarks of their respective owners.

Free and Open-Source Licensing Notices

If applicable, Free and Open-Source Software (FOSS) licensing notices are available in the product installation.

Third-Party Links

Any links to third-party websites included in this document are for your convenience only. Synopsys does not endorse and is not responsible for such websites and their practices, including privacy practices, availability, and content.

www.synopsys.com

Contents

Customer Support	4
Synopsys Statement on Inclusivity and Diversity	4

1. Introduction	6
------------------------------	----------

2. Assertion Verification Acceleration Usage Model	8
Setting Up the Environment	8
The Flow	8
Preparing the Inputs	10
The Waveform (FSDB)	10
SVA/AIP Source File and the Corresponding VCS Compilation Script	10
The Bind File	10
IDX Assertion Verification Acceleration Configuration File	11
Running the IDX Assertion Testbench Compilation	13
Running the IDX Waveform Replay Simulation	15
Debugging the SVA/AIP Simulation Results	15
Improving the Performance of SVA Compilation Flow	15
Executing an Example	17
SVA Compilation Flow (idx -compile_sva)	17
AIP Compilation Flow (idx -compile_aip)	19
Limitations and Known Issues	21

Preface

IDX enables reusing waveforms to avoid continually re-creating and re-running stimulus. As the waveform (usually FSDB) records the values, each signal/bus contains throughout the simulation run, it is a good source to 'replay' the values as the stimulus in a simulation.

Customer Support

For any online access to the self-help resources, you can refer to the documentation and searchable knowledge base available in SolvNetPlus.

To obtain support for your Verdi product, choose one of the following:

- Open a case through SolvNetPlus.

Go to <https://solvnetsynopsys.com/s/contactsupport> and provide the requested information, including:

- Product L1 as Verdi
- Case Type

Fill in the remaining fields according to your environment and issue.

- Send an e-mail message to support_center@synopsys.com.

Include product name (L1), sub-product name/technology (L2), and product version in your e-mail, so it can be routed correctly.

Your e-mail will be acknowledged by automatic reply and assigned a Case number along with Case reference ID in the subject (ref:____:ref).

For any further communication on this Case via e-mail, send e-mail to support_center@synopsys.com and ensure to have the same Case ref ID in the subject header or else it will open duplicate cases.

Note:

In general, we need to be able to reproduce the problem in order to fix it, so a simple model demonstrating the error is the most effective way for us to identify the bug. If that is not possible, then provide a detailed explanation of the problem along with complete error and corresponding code, if any/permissible.

Synopsys Statement on Inclusivity and Diversity

Synopsys is committed to creating an inclusive environment where every employee, customer, and partner feels welcomed. We are reviewing and removing exclusionary

Contents

language from our products and supporting customer-facing collateral. Our effort also includes internal initiatives to remove biased language from our engineering and working environment, including terms that are embedded in our software and IPs. At the same time, we are working to ensure that our web content and software applications are usable to people of varying abilities. You may still find examples of non-inclusive language in our software or documentation as our IPs implement industry-standard specifications that are currently under review to remove exclusionary language.

1

Introduction

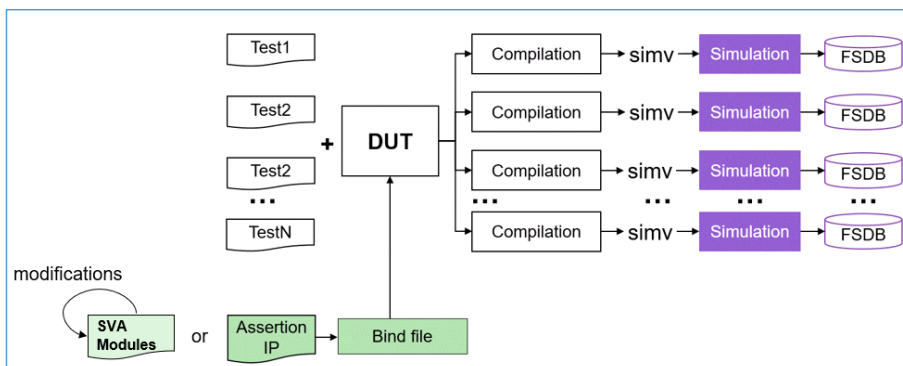
SystemVerilog Assertions (SVA) are used to describe either legal or illegal behavior that can be observed at the point of their insertion by simulation or formal verification, and then flagged as having passed or failed. The benefits of assertions fall into the following categories: reducing verification time, catching errors earlier, focusing the design effort, and pinpointing sources of error.

Assertion IPs (AIP) have pre-built assertions that help to verify and signoff the IPs in System on Chip (SoC) designs. They can significantly reduce verification effort and improve the design quality by monitoring a target design without modifying the corresponding RTL, and they can be applied in either simulation or formal verification.

In the simulation environment, it is quite complex to compile the Testbenches and the RTL. Therefore, once the existing simulation environment is ready, applying the SVA/AIP afterwards become a challenge. The following figure illustrates that there are three issues that you need to overcome:

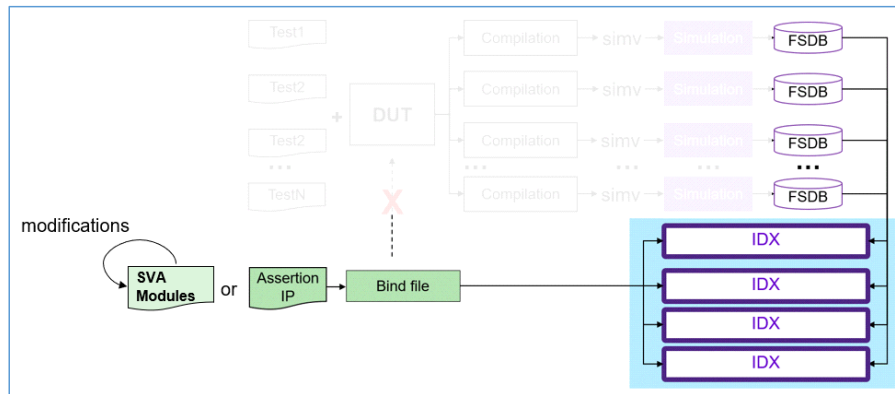
- The SVA/AIP must be bound to the DUT; an additional bind file must be provided.
- The recompilation is needed after applying the SVA/AIP and the bind file; this may introduce unexpected errors due to the complex existing simulation environment.
- The simulation runtime performance may be downgraded due to the additional SVA/AIP checkers.

Figure 1 Three issues to Overcome



To efficiently adopt SVA/AIP to the existing simulation environments, IDX offers the **Assertion Verification Acceleration** flow which is shown in the following figure:

Figure 2 *Assertion Verification Acceleration Flow*



2

Assertion Verification Acceleration Usage Model

The following sections describe how to enable IDX Assertion Verification Acceleration flow and gives a demo case to show how it works.

- [Setting Up the Environment](#)
- [The Flow](#)
- [Limitations and Known Issues](#)

Setting Up the Environment

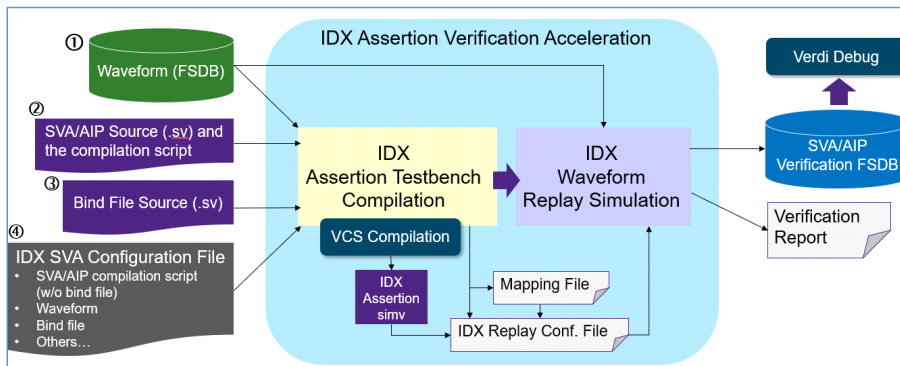
You need to run the following command in the C (CSH) shell environment before running IDX:

```
% setenv VERDI_HOME <verdi_installation_path>
% setenv PATH $VERDI_HOME/bin:$PATH
```

The Flow

The following figure illustrates the flow of IDX assertion verification acceleration:

Figure 3 *IDX Assertion Verification Acceleration*



As the flow illustrates, IDX assertion verification acceleration flow consists of the following two steps:

Step 1: IDX assertion testbench compilation.

IDX contains two usage flows for this step. You can choose any one of the flows to fit your requirement.

- `idx -compile_sva flow`
- `idx -compile_aip flow`

Step 2: IDX waveform replay simulation

The differences between the `idx -compile_sva` and the `idx -compile_aip` flows are:

- Whether DUT (compiled KDB) is required:
 - AIP, as the name suggests, does not contain the external reference (XMR) inside. Therefore, IDX can generate the assertion testbench without considering the DUT. The `idx -compile_aip` flow does not require the compiled KDB of the DUT.
 - SVA modules can contain the XMR. In the `idx -compile_sva` flow, to successfully compile the SVA modules, IDX requires the compiled KDB to extract the types and the signals in the DUT.
- Whether specifying the bind file is required:
 - The `idx -compile_aip` flow requires you to specify the bind file to let IDX understand the connections between AIP and the DUT.
 - The `idx -compile_sva` flow does not require you to specify the bind file. IDX can extract the bind information from the KDB.

The table below outlines the differences between the `idx -compile_sva` and the `idx -compile_aip` flows to generate the assertion testbench:

	Usage Model	Supports XMR in Assertion	Requires KDB of DUT	Requires Specifying Bind File
<code>idx -compile_sva</code> <code>idx_sva_comp.cfg</code>	SVA replay	Yes	Yes	No
<code>idx -compile_aip</code> <code>idx_aip_comp.cfg</code>	AIP replay	No	No	Yes

The detailed usage of above steps is described in the following sections.

Preparing the Inputs

The IDX requires four inputs to enable the AIP verification flow:

1. The waveform
2. The SVA/AIP source file and the corresponding VCS compilation script
3. (For the `idx -compile_aip` flow) The bind file that is used to bind the AIP to the simulation environment; only one bind file is allowed
4. The configuration file for IDX assertion verification acceleration flow

The Waveform (FSDB)

IDX allows to read the waveforms, that is, FSDB generated by any of the tools:

Note:

The assertions are not necessary to be included in the source waveform used for assertion debug.

SVA/AIP Source File and the Corresponding VCS Compilation Script

The SVA/AIP source files should be provided. An alternative is to specify these files with the `-f` option. In addition, the corresponding VCS compilation script is necessary. Some SVA related options like `-assert enable_diag` are strongly suggested to add to the script.

The Bind File

The bind file means, a SystemVerilog file which contains the "bind" statements to bind the SVA/AIP to the design. Here the design hierarchies and the design signals must be existent in the waveform. Note that only one bind file is allowed in the IDX assertion verification acceleration flow.

The syntax for bind statement is as follows:

```
bind <inst_name | module> SVA_module #[SVA_parameters] SVA_inst_name  
[SVA_ports]
```

The `<inst_name>` is the design full hierarchical name in the waveform. The `<module>` is the module definition name. IDX supports only one `<inst_name | module>` in a bind statement.

For example, given the design below:

```
module top (...);  
  DUT u_dut_01 ( .* );  
  DUT u_dut_02 ( .* );
```

```
TEST u_test (...);  
...  
endmodule  
  
module DUT (...);  
...  
endmodule  
  
module TEST (...);  
...  
endmodule
```

IDX supports the bind file with below description:

```
// bind <module> ...  
bind DUT SVA_MOD #(.p(1'b0)) u_sva_checker  
  (.clk(clk_i), .rst, .addr);
```

or

```
// bind <inst_name> ...  
bind top.u_dut_01 SVA_MOD #(.p(1'b0)) u_sva_checker  
  (.clk(clk_i), .rst, .addr);
```

The following// is not supported:

```
bind top.u_dut_01, top.u_dut_02 SVA_MOD #(.p(1'b0)) u_sva_checker  
  (.clk(clk_i), .rst, .addr);  
  
// not supported  
bind DUT, TEST SVA_MOD #(.p(1'b0)) u_sva_checker  
  (.clk(clk_i), .rst, .addr); multiple bind instances or modules are not  
  supported:
```

Refer to [Limitations and Known Issues](#) for more limitations about the bind file description.

IDX Assertion Verification Acceleration Configuration File

IDX requires a configuration file to describe the above inputs.

Compile SVA Flow (idx -compile_sva idx_sva_comp.cfg)

Below is the format of the idx_sva_comp.cfg configuration file in the idx -compile_sva flow:

```
//waveform file abs. path.  
WAVEFORM = $PWD/novas.fsdb  
  
//SVA source files and the compilation script  
SVA_FILE = -f $PWD/checker_file.f
```

```
//Specifying the -f file in the configuration file moves the running
//directory. Therefore, it becomes necessary to provide either the
//absolute path or the root directory along with the relative path.

COMPILE_OPT = +define+DUT_TOP=system.i_cpu.i_ALUB -sverilog
              -timescale=1ns/1ns -assert enable_diag -kdb -debug_acc+all

//SVA top scope used in the DUT hierarchy
SVA_BIND_SCOPE = system.i_cpu.i_ALUB
SVA_TOP_MOD     = alu_checker
SVA_TOP_INST    = u_checker

//run time simulation options
SIMV_OPT = -l replay_aip.log -assert report=sva.report
            +fsdbfile+replay_aip.fsdb +fsdb+sva_success +fsdb+sva_vacuous

//waveform dumping Tcl commands. Use semicolon to separate tasks
//Optional)
WAVEFORM_DUMPING_TASK = fsdbDumpvars +all; fsdbDumpSVA
```

Note:

The **SIMV_OPT** and **WAVEFORM_DUMPING_TASK** attributes are the placeholder for IDX runtime simulation. To enable the assertion verification in the VCS simulation, the VCS runtime options like **-assert success** are suggested to be used. Similarly, to enable the assertion debugging in Verdi, the FSDB dumping command **fsdbDumpSVA** is also suggested.

Compile AIP Flow (idx -compile_aip idx_aip_comp.cfg)

Below is the format of the **idx_aip_comp.cfg** configuration file used in the **idx -compile_aip** flow:

```
//waveform file abs. path.
WAVEFORM = $PWD/novas.fsdb

//bind file abs. path.
BIND_FILE = $PWD/bind.sv

//SVA/AIP source files and the compilation script
COMPILE_OPT = -sverilog -f $PWD/../../src/run.f -assert enable_diag -kdb
              -debug_acc+all

//run time simulation options
SIMV_OPT = -l replay_aip.log -assert success+report+summary+vacuous
            +fsdbfile+replay_aip.fsdb +fsdb+sva_success +fsdb+sva_vacuous

//waveform dumping Tcl commands. Use semicolon to separate
//tasks (Optional)
WAVEFORM_DUMPING_TASK = fsdbDumpvars +all; fsdbDumpSVA
```

Note:

The attributes `SIMV_OPT` and `WAVEFORM_DUMPING_TASK` are the placeholder for IDX runtime simulation. To enable the assertion verification in the VCS simulation, the VCS runtime options like `-assert success` are suggested to be used. Similarly, to enable the assertion debugging in Verdi, the FSDB dumping command `fsdbDumpSVA` is also suggested.

Attribute	Description
+in	Maps the input ports of the scope. For example, <code>AUTO_MAP = +in</code>
+out	Maps the output ports of the scope. For example, <code>AUTO_MAP = +out</code>
+inout	Maps the inout ports of the scope. For example, <code>AUTO_MAP = +inout</code>
+mda	Maps the multi-dimensional array signals under the scope. For example, <code>AUTO_MAP = +mda</code>
+reg	Maps the registers under the scope. For example, <code>AUTO_MAP = +reg</code>

Running the IDX Assertion Testbench Compilation

Perform the following command to run IDX assertion testbench compilation:

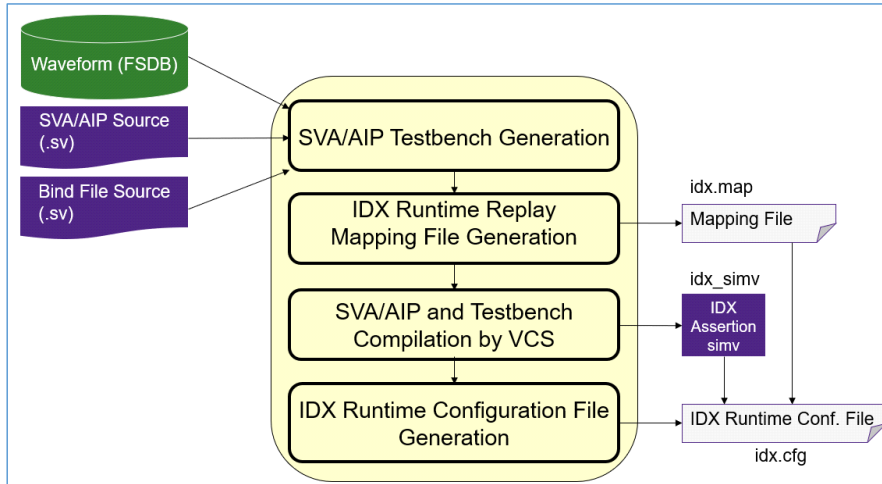
```
% idx -compile_sva <SVA_compilation_config> [-o
<SVA_compilation_directory_name>]
```

or

```
% idx -compile_aip <AIP_compilation_config> [-o
<AIP_compilation_directory_name>]
```

The command executes the steps inside, as the flow diagram illustrates:

Figure 4 Running IDX Assertion Testbench



If the option `-o <SVA_compilation_directory_name>` is not specified in the `idx -compile_sva` flow, IDX uses the default directory name `idx.sva.dbdir/`.

Similarly, if the option `-o <AIP_compilation_directory_name>` is not specified in the `idx -compile_aip` flow, IDX uses the default directory name `idx.aip.dbdir/`.

After finishing the step, the IDX assertion `simv` executable will be generated, and IDX gives the following messages:

idx -compile_sva Flow

```
[IDX_INFO]SVA compilation was performed successfully. Refer to config
file '/remote/work_IDX/idx.sva.dbdir/idx.cfg' for more detail info.
```

```
[IDX_INFO]Please use the command below to run the replay process:
```

```
> idx -cfg /remote/work_IDX/idx.sva.dbdir/idx.cfg -work
WORK_DIR_NAME_YOU_WANT
```

```
Or
```

```
> /remote/work_IDX/run_idx WORK_DIR_NAME_YOU_WANT
```

```
[IDX_INFO]Compile SVA done.
```

idx -compile_aip Flow

```
[IDX_INFO]AIP compilation was performed successfully. Please use the
command below to do the replay:
```

```
> idx -cfg /tmp/guest/idx.aip.dbdir/idx.cfg -work WORK_DIR_NAME
```

```
Refer to config file '/tmp/guest/idx.aip.dbdir/idx.cfg' for more detail
info.
```

```
[IDX_INFO]Compilation AIP done.
```

The message instructs how to proceed the follow-up IDX waveform replay simulation.

Running the IDX Waveform Replay Simulation

After finishing IDX assertion testbench compilation described in [Running the IDX Assertion Testbench Compilation](#), all the necessary files are available and described in the generated configuration file for IDX waveform replay simulation. Therefore, just follow the message to perform the waveform replay simulation.

idx -compile_sva Flow

```
% idx -cfg ./idx.sva.dbdir/idx.cfg -work <run_time_work_dir>
```

idx -compile_aip Flow

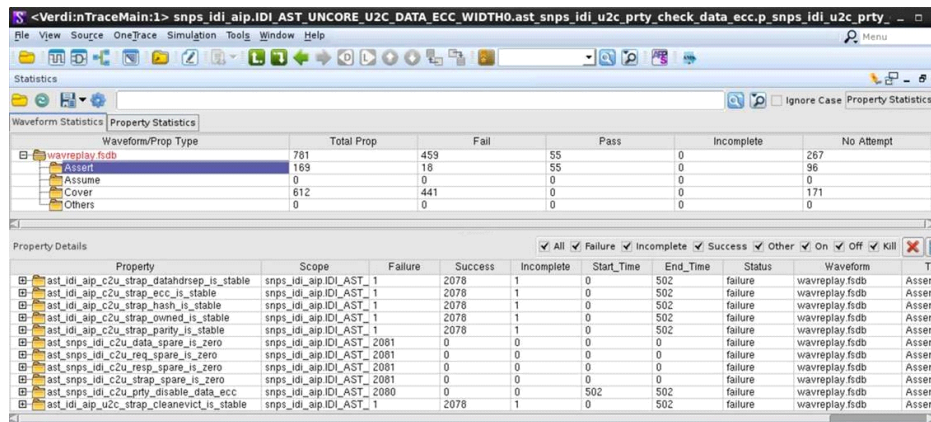
```
% idx -cfg ./idx.aip.dbdir/idx.cfg -work <run_time_work_dir>
```

Refer to the *IDX Waveform Replay Simulation* documentation to learn more about IDX runtime simulation usages.

Debugging the SVA/AIP Simulation Results

Once the replayed FSDB is generated, we can use Verdi to debug the SVA/AIP simulation results. In Verdi, open **Tools > Property Tools > Statistics** to check the assertion simulation results. The following is a snapshot in Verdi:

Figure 5 Debugging the SVA/AIP Simulation Results



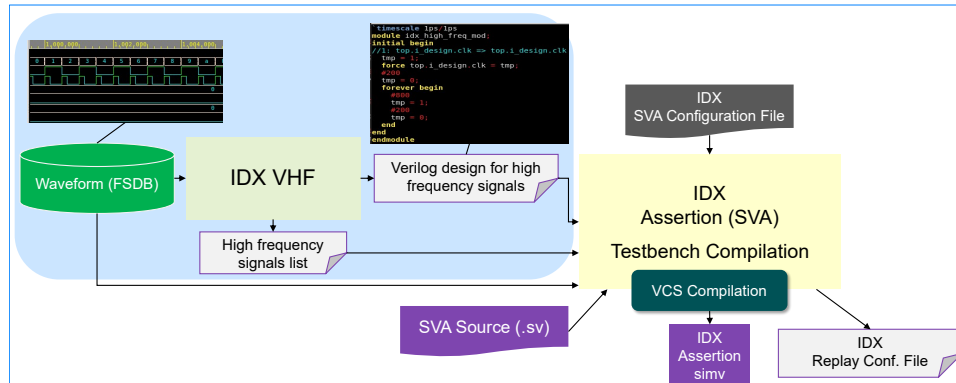
Improving the Performance of SVA Compilation Flow

Replaying the value of the high frequency signals, for example, clock signals could take a significant portion of the replay simulation runtime. IDX proposes the VHF (Verilog

code to handle High Frequency signals) flow for replaying such signals in SVA to achieve performance similar to that of a native simulation.

The following figure illustrates the IDX VHF flow:

Figure 6 IDX VHF Flow



As the flow illustrates, IDX VHF analyzes the attributes of the signals, that is, the shape, duty cycle, frequency, and so on, from the waveform. It then generates the Verilog design for high frequency signals.

The IDX SVA compilation flow elaborates the Verilog design for high frequency signals - with SVA checkers and DUT skeleton together - to generate the following:

- a new IDX assertion simulation binary
- the configuration file for IDX waveform replay simulation later

Steps to Enable the IDX VHF Flow

To enable the IDX VHF flow, set the `% setenv REP_ENABLE_VHF_ANALYSIS 1` environment variable prior to running the SVA compilation flow.

Limitation of the IDX VHF Flow

Currently, the VHF flow can support periodic signals only. VHF does not support irregular signals such as gated clock and frequency scaling signals.

Executing an Example

SVA Compilation Flow (idx -compile_sva)

The Design

We are using a simple CPU design and the SVA checkers as the example. The design case is located at the following path:

```
$VERDI_HOME/demo/IDX/SVA_Debug
```

The file structure in the directory is as follows:

- `Design/`: This is where the CPU design and the simulation testbench are located.
- `SVA_Checker/`: This is where the SVA checker files are located.
- `run_simulation_all.f`: The run file for TB, DUT, and SVA.
- `checker_file.f`: The file list for SVA modules.
- `dump.tcl`: The UCLI command to dump FSDB.
- `idx_compile_sva.cfg`: This is the configuration file for IDX `idx -compile_sva` compilation.
- `sourceMe`: The environment setting.

Running the Simulation Environment

Execute the following command to run the DUT simulation and generate the corresponding KDB and FSDB:

```
% source sourceMe  
% make compile_design_and_checker
```

The FSDB file `rtl.fsdb` is generated. The KDB inside the `simv_design_checker.daidir` is generated as well.

Running the IDX Assertion Testbench Compilation

The configuration file `idx_compile_sva.cfg` is as follows:

```
#  
# IDX compile_sva inputs  
#  
WAVEFORM = ./rtl.fsdb  
SVA_FILE = -f checker_file.f  
COMPILE_OPT = +define+DUT_TOP=system.i_cpu.i_ALUB -kdb -sverilog -assert  
svaext -assert enable_diag -timescale=1ns/1ns  
  
SVA_BIND_SCOPE = system.i_cpu.i_ALUB
```

```
SVA_TOP_MOD      = alu_checker
SVA_TOP_INST     = u_checker

#
# IDX run time option
#
SIMV_OPT          = -l idx_sva_sim.log -assert
                  report=idx_sva.report -assert summary+vacuous
WAVEFORM_DUMPING_TASK = fsdbDumpSVA ; fsdbDumpvars +all
```

Execute the following command to generate the IDX SVA simv executable:

```
% make idx_compile_sva
```

or

```
% idx -compile_sva idx_compile_sva.cfg -o SVA_COMPILE
```

The necessary files are generated in the output folder `SVA_COMPILE.sva.dbdir/`, for the follow-up IDX runtime simulation. Refer to the content of the file `SVA_COMPILE.sva.dbdir/idx.cfg` for details.

Running the IDX Waveform Replay Simulation

Follow the message after IDX assertion testbench compilation, or execute the following command for IDX waveform replay simulation:

```
% make idx_run_sva
```

or

```
% idx -cfg ./SVA_COMPILE.sva.dbdir/idx.cfg -work SVA_REPLAY
```

Finally, the SVA checker generates the following assertion report `idx_sva.report`:

idx_sva.report

```
===== Generating Assertion Summary =====

"system.i_cpu.i_ALUB.u_checker.OF_COVER", 122 attempts, 2 matches, 120
mismatches, 0 incompletes
"system.i_cpu.i_ALUB.u_checker.genblk[0].cv", 13 attempts, 3 matches, 10
mismatches, 0 incompletes
"system.i_cpu.i_ALUB.u_checker.genblk[1].cv", 13 attempts, 5 matches, 8
mismatches, 0 incompletes
"system.i_cpu.i_ALUB.u_checker.genblk[2].cv", 13 attempts, 4 matches, 9
mismatches, 0 incompletes
"system.i_cpu.i_ALUB.u_checker.genblk[3].cv", 13 attempts, 2 matches, 11
mismatches, 0 incompletes
"system.i_cpu.i_ALUB.u_checker.genblk[4].cv", 13 attempts, 2 matches, 11
mismatches, 0 incompletes
"system.i_cpu.i_ALUB.u_checker.genblk[5].cv", 13 attempts, 1 matches, 12
mismatches, 0 incompletes
```

```
"system.i_cpu.i_ALUB.u_checker.genblk[6].cv", 13 attempts, 2 matches, 11
mismatches, 0 incompletes
"system.i_cpu.i_ALUB.u_checker.genblk[7].cv", 13 attempts, 1 matches, 12
mismatches, 0 incompletes
"system.i_cpu.i_ALUB.u_checker.ALU_SUB", 122 attempts, 116 successes, 6
failures, 0 incompletes
"system.i_cpu.i_ALUB.u_checker.u_checker_sub.ALU_SUB_HIER", 13 attempts,
9 successes, 4 failures, 0 incompletes
Summary: 2 assertions, 2 with attempts, 2 with failures
          9 covers, 9 with attempts
```

```
===== End Assertion Summary =====
```

AIP Compilation Flow (idx -compile_aip)

The Design

We are using a simple ALU design and the corresponding ALU Assertion checker IP as the design example. The design case is located at below path:

```
$VERDI_HOME/demo/IDX/AIP_Debug
```

The file structure in the directory is as follows:

- `Simulation_Env/`: This is where the ALU design and the simulation testbench are located.
- `AIP_src/`: This is where the ALU Assertion checker IP is located.
- `Bind_File/`: This is where the bind file that binds the ALU AIP to the ALU simulation environment is located.
- `aip_comp.cfg`: This is the configuration file for IDX AIP compilation.

Run the Simulation Environment

Execute the following command to run the ALU simulation and generate the corresponding FSDB:

```
% make simulation_verification
```

The FSDB file, `validation.fsdb` is generated in the `Simulation_Env/` directory.

Running IDX Assertion Testbench Compilation

The configuration file `aip_comp.cfg` is as follows:

```
//waveform file abs. path.
WAVEFORM = ./Simulation_Env/validation.fsdb

//bind file abs. path.
BIND_FILE = ./Bind_File/alu_bind.sv
```

```
//AIP source files and the compilation script
COMPILE_OPT = -sverilog ./AIP_src/alu_checker.sv -assert enable_diag -kdb
               -debug_acc+all

//run time simulation options
SIMV_OPT = -l replay_aip.log -assert success+report+summary+vacous
            +fsdbfile+replay_aip.fsdb +fsdb+sva_success +fsdb+sva_vacuous

//waveform dumping Tcl commands. Use semicolon to separate
tasks (Optional)
WAVEFORM_DUMPING_TASK = fsdbDumpvars +all; fsdbDumpSVA
```

Execute the following command to generate the AIP simv executable:

```
% make compile_aip
```

or

```
% idx -compile_aip aip_comp.cfg -o AIP_COMPILE
```

The `simv` executable `simv_idx_aip` is generated. In addition, the necessary files are also generated in the output folder `AIP_COMPILE.aip.dbdir/`, for the follow-up IDX runtime simulation. Refer to the content of the file `AIP_COMPILE.aip.dbdir/idx.cfg` for details.

Running IDX Waveform Replay Simulation

Follow the message after IDX assertion testbench compilation, or execute the following command for IDX waveform replay simulation:

```
% make replay_aip
```

or

```
% idx -cfg ./AIP_COMPILE.aip.dbdir/idx.cfg -work AIP_REPLAY
```

Finally, the AIP checker generates the following simulation messages after the replay simulation:

```
===== Generating Assertion Summary =====

"idx_wrapper_1_alu_tb_top_dut.check_inst.a_prop_rst", 17 attempts, 17
successes, 0 failures, 0 incompletes
"idx_wrapper_1_alu_tb_top_dut.check_inst.a_prop_valid_sum", 17 attempts,
16 successes, 1 failures, 0 incompletes
"idx_wrapper_1_alu_tb_top_dut.check_inst.a_prop_alu_op", 17 attempts, 12
successes, 5 failures, 0 incompletes

===== End Assertion Summary =====
```

Limitations and Known Issues

The limitations of IDX assertion verification acceleration flow are listed as follows:

- IDX AIP compilation flow does not support Mixed-language/VHDL design.
- Only one bind file is allowed in the IDX AIP compilation flow. If there are multiple bind files, it is suggested to concatenate them into a single bind file.
- In the bind file, below port mapping style are not supported:

- Positional port map

- The following example illustrates the unsupported port mapping style:

```
// not supported - positional port mapping
bind DUT SVA_MOD u_sva_checker ( clk_i, rst, addr );
```

- If the signal used in the bind file is of unpacked type, for example, consider the following AIP example:

```
module SVA_CHECKER ( input i_clk, ... );
logic i_clk [1:0]; // unpacked type1
...
endmodule
```

- It may cause VCS compilation failure if the bind file is written as shown in the following example:

```
// not supported - unpacked port mapping
bind DUT SVA_CHECKER u_sva_checker ( .i_clk (i_clk), ...);
```

To resolve the issue, you need to split the unpacked bus to the individual bit-wise signal, for example:

```
// supported - concatenation port mapping
bind DUT SVA_CHECKER u_sva_checker ( .i_clk ( {i_clk[0], i_clk[1]} ),
...);
```